

Helix Thready — Hetzner Host Provisioning

Table of Contents

1. Host baseline (sizing)
 - 1.1 Candidate Hetzner SKUs + object-tier split
2. Provisioning phases diagram
3. Phase A — root bootstrap (one time)
4. The privileged-port sysctl (why root is needed once)
5. Firewall
6. Phase B — the thready user (rootless)
7. Phase C — first deploy
8. Host toolchain (incl. docs_chain deps)
9. Verified vs assumed
10. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (deployment)	Initial root→thready provisioning, rootless Podman, firewall, linger, sysctl, first deploy
2	2026-07-21	swarm (deployment review)	Split the provisioning-phases prose into multiple paragraphs
3	2026-07-22	swarm (deployment)	Pass 3: added the candidate Hetzner SKU + object-tier split table (§1.1) to make [OPEN: host-sizing] implementation-ready; cross-linked the day-2 operations runbook
4	2026-07-22	swarm (docs export)	Fixed inline mermaid syntax so diagram renders

This document is the step-by-step provisioning of the single Hetzner dedicated host: from **root** bootstrap to the unprivileged **thready** user under which all three environments run rootless (§14.2 , Q8). It ends at a first health-gated deploy.

Diagram source: sibling under [diagrams/](#) . Rendered PNG/SVG exported via Docs Chain (§11.4.65).

Table of Contents

1. Host baseline (sizing)
 - 1.1 Candidate Hetzner SKUs + object-tier split
2. Provisioning phases diagram
3. Phase A — root bootstrap (one time)
4. The privileged-port sysctl (why root is needed once)
5. Firewall
6. Phase B — the thready user (rootless)
7. Phase C — first deploy
8. Host toolchain (incl. docs_chain deps)
9. Verified vs assumed
10. Open items

1. Host baseline (sizing)

`[DEFAULT — adjustable]` baseline for the Large scale (Q5) — three co-resident stacks + object tier:

Resource	Baseline	Rationale
vCPU	≥ 16	prod + sta + dev app/data planes co-reside
RAM	≥ 64 GB	Postgres + pgvector working set; JetStream; three stacks
Disk	NVMe for Postgres/pgvector; large HDD/NVMe pool for MinIO (50 TB+)	Q5, asset tier
Network	1 Gbit+; static IPv4 + IPv6	edge on one IP; HTTP/3 over UDP
GPU	not on this host — HelixLLM runs on the workstation/GPU node	Q5; reached as external endpoint

`[OPEN: host-sizing]` — exact Hetzner SKU (e.g. a dedicated **AX/EX** line box) and whether MinIO uses local disks or a Hetzner volume/Storage Box for the object tier is finalized after load tests. HelixLLM's GPU node topology is separate. The candidate mapping in §1.1 makes this implementation-ready without pre-committing the operator to a purchase.

1.1 Candidate Hetzner SKUs + object-tier split

`[RESEARCH]` `[DEFAULT — adjustable]` — mapping the abstract baseline (≥ 16 vCPU / ≥ 64 GB, NVMe for the DB, a large pool for 50 TB+ assets) onto concrete, purchasable Hetzner **dedicated** hardware. The central

sizing tension is that **fast compute** (AX/EX line, NVMe) and **bulk storage** (SX line, many large HDDs) live in different product families — so the decision is really “one box or two tiers”:

Option	Family / representative SKU	Compute	RAM	Storage profile	Fit
A — single compute box + external object tier <i>(recommended default)</i>	AX line (e.g. AX102-class, Ryzen 9, 16C/32T)	16C/32T	128 GB DDR5	2× ~2 TB NVMe (RAID1) for DB + hot state	Best DB/pgvector latency; assets go to a separate object tier (below)
B — single storage box	SX line (e.g. SX-class storage server)	8–16C	64–128 GB	many large HDDs (tens of TB) + small NVMe	Holds 50 TB+ assets <i>on-box</i> but HDD-backed Postgres hurts pgvector latency
C — split: AX compute + SX/Storage-Box object tier	AX compute + SX server or Storage Box	16C+/32T+	128 GB+	NVMe DB on AX; HDD object pool off-box	Cleanest at scale; object tier doubles as the backup secondary candidate

Object-tier decision (the 50 TB+ MinIO volume). Three backings, in order of preference:

1. **Hetzner dedicated storage server (SX)** or a **large NVMe/HDD RAID** on the same host — highest MinIO throughput, simplest network path; best when assets and compute can share one box (Option B) or a second SX box (Option C).
2. **Hetzner Volume** attached to the AX box — elastic, but capped per-volume and network-latency bound; fine for a smaller-than-50 TB start that grows.

3. **Hetzner Storage Box** (SFTP/rc1one) — cheapest, off-host; already the **backup secondary default**, so reusing it as the live object tier would couple primary and backup — **avoid** for the live tier, keep it for backups.

Recommendation: start **Option A** (one AX compute box, NVMe DB, assets on a Volume or a small SX tier) and split to **Option C** once asset volume approaches the 50 TB envelope — the compose files and port plan do not change, only the MinIO volume's backing does. The GPU that HelixLLM needs is **not** on this host under any option; it stays the operator workstation / GPU node reached as an **external endpoint**. Exact SKU codes drift with Hetzner's catalogue, so the *family + profile* is what is fixed here; the specific model is confirmed at purchase after the load test sets the real asset-growth and pgvector-working-set curves.

2. Provisioning phases diagram

Phase A — as root (one time)

Update OS + install
podman, podman-compose,
rclone, openssl, curl,
pandoc, weasyprint



Create user 'thready' home
/home/thready



Set /etc/subuid /etc/subgid
for thready



sysctl
net.ipv4.ip_unprivileged_po
rt_start=80



Explanation (for readers/models that cannot see the diagram). Provisioning has three phases. **Phase A (root, one time)** does everything that genuinely needs privilege and nothing more: it updates the OS and installs the container + backup + cert + docs toolchain; creates the unprivileged `thready` user with home `/home/thready`; assigns the subordinate UID/GID ranges rootless Podman needs (`/etc/subuid` , `/etc/subgid`); lowers the `net.ipv4.ip_unprivileged_port_start` sysctl to 80 so a rootless container can bind 80/443; hardens SSH to key-only with no root login; installs an `nftables` firewall that allows only 22/80/443; and enables **lingering** for `thready` so the user's systemd (stacks + cert timers) runs even when nobody is logged in. After Phase A, root is **no longer used** in the runtime path.

Phase B (as `thready` , rootless) signs in with an SSH key, clones the `helix_thready` repo plus the `containers` and `lets_encrypt` submodules, drops the runtime secrets (`.env` , `api_keys.sh`) with `chmod 600` from the private repo, logs into any container registries, installs `acme.sh` rootless via the `lets_encrypt` setup script, and installs the per-environment renewal timers. Every command in this phase runs as the unprivileged user — there is no `sudo` anywhere in it.

Phase C (first deploy) issues certificates against the staging CA then production per subdomain, deploys each environment through the health-gated deploy, and enables the stacks' `systemd --user` units so they survive reboot. The arrows enforce order: linger before the user phase, timers before certs, certs before the app deploy — so nothing is attempted before its prerequisite exists.

3. Phase A — root bootstrap (one time)

Run **once** as root on a fresh Hetzner host. Idempotent where possible.

```
#!/usr/bin/env bash
# provision-root.sh (run as root, ONE TIME)
set -euo pipefail

# 1. OS + toolchain (Debian/Ubuntu shown; adapt for the chosen distro)
apt-get update && apt-get -y upgrade
apt-get -y install podman podman-compose slirp4netns fuse-overlayfs \
    rclone openssl curl git nftables \
    pandoc weasyprint # docs_chain deps — GAP #19

# 2. Create the unprivileged runtime user
useradd -m -s /bin/bash -d /home/thready thready

# 3. Rootless Podman subordinate ID ranges (65536 each is the usual default)
usermod --add-subuids 100000-165535 --add-subgids 100000-165535 thready

# 4. Allow rootless bind of 80/443 (see §4 for the why)
echo 'net.ipv4.ip_unprivileged_port_start=80' > /etc/sysctl.d/99-thready-ports.conf
sysctl --system

# 5. Harden SSH (key-only, no root login)
install -m 700 -o thready -g thready -d /home/thready/.ssh
# (append the operator's public key to /home/thready/.ssh/authorized_keys, chmod 600)
sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
systemctl reload ssh

# 6. Firewall (see §5)
install -m 600 /dev/stdin /etc/nftables.conf <<'NFT'
# ... see §5 ...
NFT
systemctl enable --now nftables

# 7. Let the thready user's systemd run without an active login (stacks + cert timers)
loginctl enable-linger thready
```

The SSH credentials for the `thready` account are provided at deployment time and kept **only in the private submodule** (§14.2); they never enter a public repo or a log.

4. The privileged-port sysctl (why root is needed once)

Rootless containers cannot bind ports < 1024 by default. Thready needs the edge on 80 and 443. Two options; Thready picks the first as its `[DEFAULT — adjustable]`:

1. **Lower** `net.ipv4.ip_unprivileged_port_start` **to 80** (chosen) — a single, auditable sysctl set once by root in Phase A. Thereafter the rootless `thready` edge binds 80/443 with **no** runtime privilege. Clean and simple.

2. **Port-forward helper / socket activation** — keep the sysctl at 1024 and forward 80/443 via a privileged helper or `systemd` socket. More moving parts; avoided for MVP.

Only the edge needs this; every other service binds a high loopback port in its `band`. This keeps the privileged-port surface to exactly two ports on one container.

5. Firewall

Minimal ingress — the entire public surface is SSH + the edge:

```
# /etc/nftables.conf
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    iif "lo" accept
    ip protocol icmp accept
    ip6 nexthdr icmpv6 accept
    tcp dport 22 accept          # SSH (key-only)
    tcp dport { 80, 443 } accept # edge: HTTP-01 + HTTPS
    udp dport 443 accept        # HTTP/3 (QUIC) — edge
  }
  chain forward { type filter hook forward priority 0; policy drop; }
  chain output { type filter hook output priority 0; policy accept; }
}
```

- **No** database/MinIO/NATS port is ever open to the internet — they bind `127.0.0.1` in their bands and are reached only via the compose network or an operator SSH tunnel.
- `udp/443` is required for HTTP/3 (QUIC), which the edge and `vasic-digital/http3` use.
- Optional: `fail2ban` on SSH; egress stays open for ACME, image pulls and messenger APIs.

6. Phase B — the thready user (rootless)

```
#!/usr/bin/env bash
# provision-thready.sh (run as the thready user, rootless)
set -euo pipefail
cd /home/thready

# 1. Clone the project + the two deployment submodules
git clone git@github.com:HelixDevelopment/helix_thready.git repo
git -C repo submodule update --init submodules/containers submodules/lets_encrypt || true
ln -sfn repo/submodules/containers /home/thready/submodules/containers
ln -sfn repo/submodules/lets_encrypt /home/thready/submodules/lets_encrypt

# 2. Layout (see podman-compose.md §3)
mkdir -p edge/certs/{dev,sta,prod} dev/config sta/config prod/config secrets releases

# 3. Runtime secrets from the PRIVATE repo – chmod 600, never committed here
# (see secrets-and-config.md). Placeholder: operator copies them in securely.
for e in dev sta prod; do install -m 600 /dev/null "$e/.env"; done
install -m 600 /dev/null secrets/api_keys.sh

# 4. Rootless acme.sh install (per env config)
for e in prod sta dev; do
    submodules/lets_encrypt/scripts/setup.sh --config "$e/config/lets_encrypt.conf" --install
done

# 5. Install the rootless renewal timers (see tls-lets-encrypt.md §10)
# le-renew-{dev,sta,prod}.timer under ~/.config/systemd/user/
systemctl --user daemon-reload
```

- Everything runs as `thready`; no `sudo` appears anywhere in this phase.
- `podman login` (if pulling from a private registry) stores creds under the user's runtime dir.

7. Phase C — first deploy

```
# As thready. Staging certs first (rehearse), then production.
for e in prod sta dev; do
  # a. issue certs (LE_STAGING=1 → 0 after rehearsal), per subdomain
  submodules/lets_encrypt/scripts/issue.sh --config "$e/config/lets_encrypt.conf"
done

# b. bring up the edge, then each env via the health-gated deploy
cd edge && podman-compose -p thready-edge up -d && cd ..
for e in dev sta prod; do
  submodules/containers/scripts/thready-deploy.sh "$e" "THREADY-$(cat repo/VERSION)"
done

# c. enable stack + edge units so they survive reboot (linger already on)
systemctl --user enable --now thready-edge.service \
  thready-dev.service thready-sta.service thready-prod.service
```

The deploy in step (b) is the full **health-gated pipeline with rollback**; a failed env restores itself and does not block the others.

8. Host toolchain (incl. docs_chain deps)

[GAP: #19 docs_chain] — the **docs_chain** engine honestly **SKIPS** md→HTML/PDF sibling generation when **pandoc** / **weasyprint** are absent (the gap register notes the current host lacks them). Phase A therefore **installs pandoc and weasyprint** so every **.md** in this pack gets its HTML/PDF siblings per [CONSTITUTION §11.4.65]. The full host toolchain:

Tool	Purpose
podman, podman-compose, slirp4netns / pasta, fuse-overlayfs	rootless container runtime
rclone	ship backups to the secondary store
acme.sh (via lets_encrypt setup.sh)	ACME client (installed rootless in Phase B)
openssl, curl	cert validation + reachability probes (lets_encrypt deps)
pandoc, weasyprint	docs_chain md→HTML/PDF ([GAP: #19])
git	repo + submodules + all-upstreams push (§2.1)
nftables	firewall

9. Verified vs assumed

- **VERIFIED:** the root→ `thready` user model with home `/home/thready` (§14.2 , Q8); rootless mandate (§11.4.76/161); `lets_encrypt_setup.sh --install` is rootless (module README); the docs_chain pandoc/weasyprint SKIP behaviour (gap register #19); private-repo-only SSH creds (§14.2).
- **ASSUMED** / `[DEFAULT — adjustable]`: the systemctl-lowering approach for privileged ports; the Debian/Ubuntu package names; the `nftables` ruleset; the subuid/subgid ranges; the ≥16 vCPU/≥64 GB sizing baseline.

10. Open items

- `[OPEN: host-sizing]` — exact Hetzner SKU + MinIO object-tier backing (local disks vs volume vs Storage Box) + GPU-node topology, pending load tests.
 - `[OPEN: distro]` — the host distro (Debian/Ubuntu/AlmaLinux) is a `[DEFAULT — adjustable]` choice; package names in Phase A adapt accordingly.
 - `[OPEN: golden-image]` — a saved post-Phase-A image would shorten the DR runbook step 1; nice-to-have, not MVP-blocking.
-

Made with love ♥ by Helix Development.